

TD-gGA 論文再現パッケージ 設計図

コード構成・データフロー・各ファイルの責務

td_gGA プロジェクト

2026 年 7 月 8 日 (2026 年 7 月 13 日: 配布パッケージ向けにファイル名を更新)

1 全体像

注記 (2026-07-13): 本文書は開発時のワークスペース (`td_gGA_solver_paperconv.py` 等、Route A/C の名残があるファイル名) から、第三者に配布するための本パッケージ (`td_gGA_solver.py` 等、役割がわかる名前) 向けに更新したもの。ファイル名以外の物理・アルゴリズムの内容に変更はない。旧名との対応は同梱の `README.md` を参照。

本ワークスペースは Guerci–Capone–Lanatà, PRR **5**, L032023 (2023) の TD-gGA (時間依存ゴースト Gutzwiller 近似) の検証済み実装である。設計は次の 3 層に分離されている:

層	役割	成果物の置き場
静的計算	平衡 gGA 解 (初期条件の材料)	<code>static_cache/B{B}_Ui{...}_T{...}.npz</code>
時間発展	クエンチ後の TDVP 積分	<code>data/B{B}/quench_Ui{...}_Uf{...}.npz</code>
プロット	保存データの可視化のみ	<code>*.png</code>

各層は独立に再実行できる。静的解はキャッシュされ ($B = 5$ で約 25 分、 $B = 7$ で数時間の計算が 1 回きりになる)、プロットは再計算なしに何度でも描き直せる。

2 物理の要点 (実装が解いている方程式)

コードの規約 ($n(\omega)$ は標準 Fermi 行列、 $\Delta_e = \langle f^\dagger f \rangle$ 電子規約) で、 $\Lambda = 0$ ゲージの複素 TDVP:

$$\Delta_e = \langle f^\dagger f \rangle_\Phi, \quad g = [\Delta_e(1 - \Delta_e)]^{1/2}, \quad (1)$$

$$R = g^{-1} \langle f^\dagger c \rangle_\Phi, \quad B = \sum_f w_f \omega_f n_f \quad (\text{複素のまま}), \quad (2)$$

$$D = \overline{g^{-1} B R}, \quad M = R D^\top + \overline{D} R^\dagger, \quad (3)$$

$$\Lambda^c = +(\text{dg} \Delta_e[M])^\top \quad (\text{Loewner 閉形式}), \quad (4)$$

$$i \partial_t \Phi = \hat{H}_{\text{emb}}[D, \Lambda^c] \Phi, \quad \partial_t n(\omega) = -i\omega [R R^\dagger, n(\omega)]. \quad (5)$$

この組は拘束 $F_2 = \|\Delta_e - \sum_f w_f n_f\|$ とエネルギーを厳密に保存する (証明は `derivation_F2_conservation_proof`. §3, §7. 鍵は Loewner 恒等式 $[\Delta, \text{dg}[M]] = [g(\Delta), M]$).

$\Lambda = 0$ ゲージ = 自由度の厳密消去 (2026-07-09 追記)

Λ はエルミート $B \times B$ 行列で、一般には B^2 個の実自由度を持つ。変分表示のうえでは Λ は物理的な観測量を変えないゲージ冗長度であり、上記 Loewner 恒等式のおかげで $\Lambda \equiv 0$ というゲージを初期時刻だけでなく全時刻にわたって厳密に維持できる (近似ではない)。この恒等式は Δ_e ($= U_i$ や状態そのもの) に依らない一般的な代数関係なので、「 U_i が小さいから許された」という話ではない (実測でも $B = 3$, $U_i = 0.05 \rightarrow U_f = 2.55$ の大クエンチで $|\Delta E/E|_{\max} = 2.3 \times 10^{-7}$)。

結果として力学変数は $(\Phi, n(\omega))$ だけになり、 Λ の B^2 自由度は毎ステップの root-find (旧 Route C 方式) なしに完全に消去される。旧 Route C が $B \geq 3$ でエネルギーを大きく破っていたのは、この消去が発見される前に Λ を独立変数として反復的に解いていたことに加え、`solve_lambda_differential` 内の符号バグ (`derivation_F2_conservation_proof.md` §6) が重なったためであり、「動的 Λ 方式が原理的に不可能」だったわけではない。ただ、上記の厳密消去が使える以上、それを使わない実装 (Route C) を維持する理由がない。

3 ソルバー核：td_gGA_solver.py

唯一の「正しい」ソルバー。主要関数と対応する式：

関数	責務
<code>solve_static(B, U_i, T)</code>	静的 gGA 平衡解 (<code>GA.optimize_selfc</code> を呼ぶ薄いラップ)
<code>save_static / load_static</code>	静的解の必要成分 ($\Phi_0, \Lambda, D, \Lambda^c, R, \text{fdaggerc}, \text{ffdagger}$) を npz へ保存／GA を最適化なしで再構成して注入
<code>get_static(B, U_i, T)</code>	キャッシュがあれば load、なければ solve して save
<code>hole_cplx(Phi, params)</code>	$\langle f_j f_i^\dagger \rangle$ (スピン平均、複素のまま。dense/sparse 自動判別)
<code>fdc_cplx(Phi, params)</code>	$\langle f^\dagger c \rangle$ (同上)
<code>g_inv_of(Delta)</code>	$[\Delta(1 - \Delta)]^{-1/2}$ (複素エルミート安全な eigh 実装)
<code>loewner_dg(Delta, M)</code>	Fréchet 微分 $\text{dg}[M]$ の Loewner 閉形式 (Λ^c 用。基底ループ・least_squares 不要)
<code>compute_RDLc_paper</code>	式 (1)–(4) : $\Phi, n \mapsto R, D, \Lambda^c, B$
<code>build_H_emb_paper</code>	$\hat{H}_{\text{emb}}$ を密行列で構築 ($B \leq 5$)
<code>apply_H_emb_sp</code>	$\hat{H}_{\text{emb}} \Phi$ を疎行列積の和で直接評価 ($B \geq 7$ 、近似なし)
<code>prepare_sparse_params</code>	演算子を疎のまま保持する軽量 params ($B \geq 6$ で自動選択)
<code>rhs_paper(t, y, params)</code>	TDVP の右辺：式 (5) (dense/sparse をディスパッチ)
<code>run(B, U_i, U_f, ...)</code>	1 本のクエンチの全工程：初期条件構築 $\rightarrow$ RK4 積分 $\rightarrow$ 観測量記録。戻り値 <code>rec</code> に $t, d, E, \Delta E/E , F_2, \sqrt{D^\dagger D}$, ghost split, eig Λ^c
<code>save_run(rec, ...)</code>	<code>rec</code> を <code>data/B{B}/</code> へ保存

run() 内の初期条件構築（重要ポイント）

1. Φ_0 = 静的 ED の基底状態 (npz 演算子基底)。 $\Delta_{e,0}, R_0$ を Φ_0 から再計算。
2. ゲージ回転 $W = \text{permmat} \cdot \text{phasemat} \cdot \text{transmat}^T$ (`fix_gauge` の変換行列) で $\Lambda_{\text{eq}} = W \Lambda_{\text{imp}} W^T$ 。
検証として $\|W R_{\text{imp}} - R_0\|$ を表示。
3. $\pm \Lambda_{\text{rot}}$ 符号自動選択：半充填の PH 対称性により (Λ, n) と $(-\Lambda, \text{PH 鏡映 } n)$ が縮退した同値レベルなので、両候補で F_2^{init} を測り小さい方を採用 ($> 10^{-3}$ なら root-refine)。 $B = 3$ は +、 $B = 5$ は - が正解だった。
4. $n_0(\omega) = f(\omega R_0 R_0^T + \Lambda_{\text{eq}})$ 、`ic_mod` フックで人工変形も可能（縮退多様体探索用）。

4 サポートモジュール（Lanata 系レガシー。変更禁止）

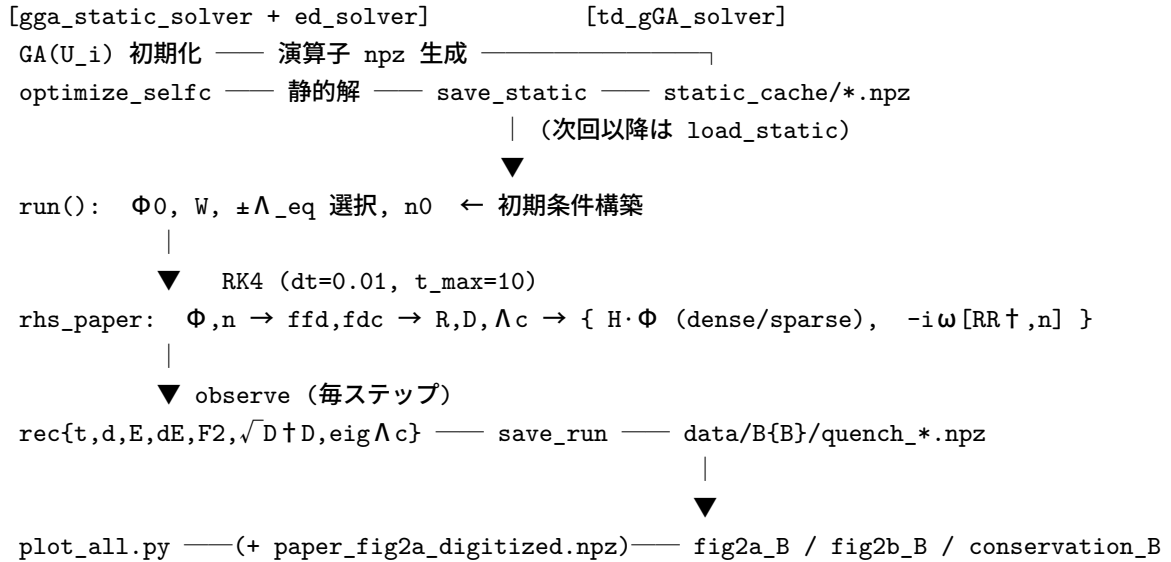
ファイル	役割
<code>gga_static_solver.py</code>	静的 gGA ソルバー本体（クラス GA）。 <code>optimize_selfc</code> (R, Λ の <code>least_squares</code>)、 <code>calc_Delta</code> ($\int \rho f(\omega R R^T + \Lambda)$)、 <code>calc_D</code> 、 <code>calc_Lmbdac</code> (実対称 Loewner)、 <code>fix_gauge</code> (Λ^c 対角化・ D 符号・並べ替えのゲージ固定)
<code>ed_solver.py</code>	埋め込みハミルトニアン of 厳密対角化。 <code>build_creation_ops</code> (Fock 空間の $f^\dagger$)、 <code>build_op_prods</code> (一体演算子積を npz 保存。 B ごとに同名上書き $\Rightarrow$ 異なる B の並走禁止)、 <code>build_Hemb</code> (疎行列で $\hat{H}_{\text{emb}}$ 組立、 $+10\hat{S}^2$ ペナルティ付き)、 <code>solve_Hemb</code> (<code>primme.eigsh</code> による疎対角化 — $B = 7$ でもそのまま動く)、 <code>calc_density_matrix</code>
<code>convenience_routines.py</code>	<code>calc_C</code> (Fermi 行列関数)、 <code>funcMat</code> 、 <code>dF</code> (Loewner)、実対称基底の生成など
<code>lattice.py</code>	半円 DOS 等の格子情報
<code>tdvp_helpers.py</code> (旧	<code>setup_frequency_grid</code> (Gauss-Legendre $\times$ 半円 DOS)、
<code>td_gGA_solver_routeA.py</code>	<code>pack_state/unpack_state</code> 、 <code>prepare_full_params</code>
+	(dense 演算子ロード、 $B \leq 5$)、 <code>compute_fdaggerc_sp</code> 、
<code>td_gGA_solver_routeC.py</code>)	<code>compute_ffdagger_sp</code> ($\langle f^\dagger f \rangle$)、 <code>solve_lambda_from_F2</code> (Λ root-finding、初期条件 Λ_{eq} の算出用)

注意：`tdvp_helpers.py` は元々 `tdvp_core.py` / `tdvp_sparse.py` という 2 ファイルで、それぞれ独立した TD ソルバー本体（開発時の呼称で Route A / Route C）だったが、規約バグ（転置・共役・符号の混在）で $B \geq 3$ の保存則・ B 依存性の再現に失敗していた。2026-07-13 の配布パッケージ整理でこの未使用のドライバ本体は削除し、`td_gGA_solver.py` が実際に `import` して使うヘルパー関数のみを残した（2 ファイル合計 1453 行 $\rightarrow$ 260 行）。統合後もファイル間の依存がゼロ（互いに `import` し合っていないかった）ことを確認したうえで、分割する意味がなくなったため 1 ファイルに統合した（260 行）。

5 実行スクリプトとプロット

ファイル	役割
run_quench.py	CLI: <code>--B 7 --dU 1.25 1.5 2.0 2.5 [--bare] [--tmax] [--dt]</code> 。get_static (キャッシュ) → run → save_run
plot_all.py	data/ を全部拾って fig2a_B.png ($d(t)$ 、論文デジタイズ点重ね)・fig2b_B.png ($\text{eig } \Lambda^c, \sqrt{D^\dagger D}$)・conservation_B.png を生成。計算ゼロ
run_B135.py	一括実行版 ($B = 1, 3, 5$ 静的 +TD+ プロット。旧形式)
run_fig2a.py, run_fig2b.py	単発の図用ランナー (plot_all 以前の形式)
compare_digitized.py	論文 Fig.2a を PDF からデジタイズし (軸は目盛りマークで校正)、我々の曲線と定量比較
run_convergence.py	$N_{\text{freq}}/\text{dt}/T$ 感度チェック
run_dU_definition.py	$U_f = U_i + \delta U$ か $U_f = \delta U$ かの判別 ($B = 1$ 周期)
saddle_scan.py	静的鞍点のシード探索 (結果: 引き込み点は単一)

6 データフロー



7 データ形式

static_cache/B{B}_Ui{U_i}_T{T}.npz : eig_vec(Φ_0), fdaggerc, ffdagger, Lmbda, D, Lmbdac, R, メタ (B, U_i, T_gGA)。

data/B{B}/quench_Ui{..}_Uf{..}.npz : t, d, E, dE($|\Delta E/E|$), F2, sqDtD, Dsplit, eigLc (形 $N_t \times B$) , メタ (B, U_i, U_f)。

`paper_fig2a_digitized.npz`: 論文 Fig.2a のデジタイズ (tg/dg_{\delta U}=B1 破線, tr/dr_{\delta U}=B3 赤)。`paper_fig2a_B57_digitized.npz`: 同 blue(B=5)/green(B=7)、 $\delta U = 2.0, 2.5$ のみ。

8 数値セッティング一覧

モデル・物理パラメータ

項目	既定値	備考
模型	単一バンド Hubbard, 半充填	$\hat{H} = \frac{U}{2} \sum_i (\hat{n}_i - 1)^2 - J \sum c^\dagger c$
格子	ベレー格子 (配位数 ∞)	半円 DOS $\rho(\omega) = \frac{2}{\pi} \sqrt{1 - \omega^2}$
エネルギー単位	半バンド幅 $D = 1$	時間単位は D^{-1} (論文と同一)
U_i	0.05	論文脚注 [55] ($U=0$ は $B \geq 3$ で縮退・不安定)
U_f	$U_i + \delta U$ (--bare で δU)	論文の δU ラベルの定義は図の精度で判別不能 ($B=1$ 周期の判別テスト参照)
温度 T	0.003	Fermi 関数の平滑化 (準 $T=0$)。感度 $\Delta d \sim 1.5 \times 10^{-5}$

時間発展 (TD)

項目	既定値	備考
積分器	固定刻み RK4	各 RHS 評価で Φ を規格化
dt	0.01	感度チェック済み ($dt=0.005$ で $\Delta d \sim 3 \times 10^{-6}$)。 $B=5$ の F_2 ドリフト ($\sim 10^{-3}$) を下げたい場合は半分に
$t_{\max}$	10	論文 Fig.2 と同範囲
N_{freq}	60	Gauss-Legendre 点 $\times \rho(\omega)$ 重み。収束確認済み (150/300 で $\Delta d \leq 1.6 \times 10^{-4}$)
状態変数	$\Phi \in \mathbb{C}^{\dim \Phi}, n(\omega) \in \mathbb{C}^{N_{\text{freq}} \times B \times B}$	複素のまま伝播 (np.real 禁止が鉄則)

静的計算・ED・初期条件

項目	既定値	備考
静的最適化	<code>least_squares</code> (R : B 個 + Λ : $B(B+1)/2$ 個)	シードは $R = [1, 0, \dots]$, Λ 係数 = $[1, 0, \dots]$ 。収束残差 $\text{Max.error} \sim 5 \times 10^{-4}$ ($B=3$) が初期条件の質の下限
ω 積分 (静的)	<code>scipy.integrate.quad</code> , <code>epsabs</code> 10^{-14}	<code>calc_Delta</code> 内
ED	<code>primme.eigsh</code> , <code>num_eig</code> =2, <code>tol</code> 10^{-12} , 'SA'	疎行列のまま。+10 $\hat{S}^2$ ペナルティで一重項基底状態を選択
EH セクター	$1+B$ フェルミオン (半充填)	$\dim \Phi = \binom{2(1+B)}{1+B}$
Λ_{eq} 構築	ゲージ回転 $W\Lambda_{\text{imp}}W^T$ + $\pm$ 符号自動選択	$F_2^{\text{init}} > 10^{-3}$ なら <code>solve_lambda_from_F2</code> (<code>tol</code> 10^{-10}) で <code>refine</code>
行列関数の保護	Δ 固有値を $[10^{-12}, 1-10^{-12}]$ にクリップ	Loewner 核の縮退判定は $ \Delta\lambda > 10^{-9}$ (それ以下は g')

到達している保存精度 (既定セッティングでの実測)

ケース ($U_i=0.05 \rightarrow U_f=2.55$, $t=10$)	$ \Delta E/E _{\text{max}}$	F_2^{max}
$B = 1$	8×10^{-10}	1.2×10^{-5} (一定)
$B = 3$	2.3×10^{-7}	2.6×10^{-5} (一定)
$B = 5$	2.8×10^{-5}	1.2×10^{-3} (RK4 離散化誤差、 dt で低減可)

9 計算コストの目安 (2026-07-13/14 実測、 $U_i = 0.05 \rightarrow U_f = 2.5$, $t_{\text{max}} = 10$)

B	$\dim \Phi$	静的 (初回のみ)	TD 1 クエンチ	$ \Delta E/E _{\text{max}}$	F_2^{max}	パス
1	6	11.7 秒	0.2 秒	3.6×10^{-7}	1.2×10^{-5}	dense
3	70	92.5 秒 (1.5 分)	2.0 秒	3.0×10^{-8}	3.6×10^{-6}	dense
5	924	2795 秒 (46.6 分)	845 秒 (14.1 分)	9.9×10^{-8}	7.4×10^{-6}	dense
7	12870	9548 秒 (159 分 $\approx$ 2.65 時間)	585 秒 (9.7 分)	2.8×10^{-8}	1.6×10^{-4}	sparse

$\dim \Phi = \binom{2(1+B)}{1+B}$ は漸近的に $\sim 4^B$ で増える (実測比 $70/6=11.7$, $924/70=13.2$, $12870/924=13.9$ と $4^2=16$ に単調接近、 B が 2 ずつ増えるため 4^2 が漸近値)。

ボトルネックは静的計算であり、TD 側ではない。 $B=7$ の静的計算が特に重い (2.65 時間) のは、§6 の断熱ラダー ($U = 0.6 \rightarrow 0.05$ を 5 段階、各段で `optimize_selfc` を 1 回ずつ) を要するため。TD 側は $B=7$ (sparse) の方が $B=5$ (dense) より速い (585 秒 vs 845 秒) という逆転が実測された——理由と一般化については次節参照。

疎行列パスは近似なし (一体演算子の Fock 行列は 1 列あたり非零 ≤ 1)。 $B=3$ で dense と厳密一致 ($\max |\Delta d| = 2 \times 10^{-13}$) を検証済み。

dense/sparse の使い分けの閾値について (2026-07-14 追記)

`run()` は既定で `sparse=(B≥6)` を使う。この閾値はメモリ制約 (dense 行列が $B = 7$ で $\sim 1.3\text{GB}$ /個になる) から決めたものであり、速度を根拠にした閾値ではない。 $\hat{H}_{\text{emb}}$ の非零率は $B = 3$ で 12.7%、 $B = 5$ で 2.1% (§6) —つまり $B = 5$ でも $\hat{H}_{\text{emb}}$ の 98% はゼロで、dense パス (`build_H_emb_paper`) はこのゼロに対しても律儀に演算している。

$B = 7(\text{sparse})$ の TD が $B = 5(\text{dense})$ より速かった実測結果を受けて、 $B = 5$ を sparse パスで動かした場合の速度を検証した：

	TD 所要時間	速度比	$d(t)$ 最大差 (dense 比)
dense (既定)	825.6 秒 (13.8 分)	—	—
sparse	6.6 秒	125×	1.2×10^{-4}

結論 (2026-07-14 実測・確定) : $B = 5$ でも sparse パスの方が **125 倍速い**。既定の `sparse=(B≥6)` は速度の観点からは不適切な閾値であり、実質的にメモリが問題にならない限り常に sparse を使うべきである ($B = 3$ 以下は $\dim \Phi$ が小さく dense でも数秒で終わるため実害はないが、 $B = 5$ では **13.8 分 vs 6.6 秒** と体感が全く変わる)。 $d(t)$ の最大差 1.2×10^{-4} は両パスが同一の $\hat{H}_{\text{emb}}\Phi$ を計算している (§6「疎行列パスは近似なし」) ことと整合的な小さな数値差で、どちらも正しい物理を再現している。今後の課題として `run()` の既定閾値 (`sparse=(B≥6)` → 例えば `sparse=(B≥3)` 相当) の見直しを推奨する。

10 運用ルール

- 異なる B の計算を同一ディレクトリで並走させない (演算子 npz が同名上書き)。
- 論文との比較は PPT 重ねでなく `compare_digitized.py` の枠組みで定量的に。
- 新しい観測量は `run()` の `observe()` に追加 → `rec` に記録 → `plot_all.py` に描画を足す、が標準手順。
- 理論・経緯はそれぞれ `derivation_F2_conservation_proof.md`・`POSTMORTEM_2026-07-07.md` を参照 (開発時の全経緯を追った `STATUS.md`・`ROADMAP.md` は本配布パッケージには含めていない。必要場合は開発ワークスペース側を参照)。

付録 A 付録: 関数リファレンス (2026-07-13 追記)

全ソースファイルの主要関数・メソッドを、役割の日本語説明と対応する数式付きで一覧する。未使用と判明したものは [未使用] と明記した (削除は行っていない・要判断のもの)。

td_gGA_solver.py (メイン時間発展ソルバー)

関数	説明・対応式
<code>hole_cplx(Phi, params)</code>	ホール行列 $\Delta_h = \langle f f^\dagger \rangle$ (スピン平均、複素のまま)

<code>fdc_cplx(Phi,params)</code>	$\langle f^\dagger c \rangle$ (複素のまま、(nq,1))
<code>_eigh_clip(Delta)</code>	Δ をエルミート化し <code>eigh</code> 、固有値を $[10^{-12}, 1-10^{-12}]$ にクリップ (数値安全化)
<code>g_inv_of(Delta)</code>	$g^{-1} = [\Delta(1-\Delta)]^{-1/2}$ (<code>_eigh_clip</code> 経由)
<code>loewner_dg(Delta,M)</code>	$g(\Delta) = \sqrt{\Delta(1-\Delta)}$ の Δ 方向 M への Fréchet 微分。Loewner 行列 $K_{ij} = \frac{g(\lambda_i) - g(\lambda_j)}{\lambda_i - \lambda_j}$ ($i = j$ or 縮退は g') を固有基底で Schur 積してから戻す
<code>compute_RDLc_paper(Phi,n_omega,params)</code>	本稿の <code>compute_RDLc</code> 。 $\Delta_e = \langle f^\dagger f \rangle$ 、 $R = g^{-1} \langle f^\dagger c \rangle$ 、 $B = \sum_f w_f \omega_f n_f$ 、 $D = \overline{g^{-1} B R}$ 、 $M = R D^\top + \overline{D} R^\dagger$ 、 $\Lambda^c = (dg_{\Delta_e}[M])^\top$ を一括計算。 呼び出し元: <code>rhs_paper()</code> (ODE の各評価) と <code>observe()</code> (各記録ステップ) の両方から。目的: 今の状態 $(\Phi, n(\omega))$ から、 $\hat{H}_{\text{emb}}$ 構築とエネルギー・ F_2 評価に必要な R, D, Λ^c をまとめて出す
<code>build_H_emb_paper(D_vec,Lmbdac,params)</code>	$\hat{H}_{\text{emb}} = \text{fdagger} + \sum_p [D_p f_p^\dagger c + \overline{D}_p c^\dagger f_p] + \sum_{pq} \Lambda_{pq}^c f_q f_p^\dagger$ を密行列で構築 ($B \leq 5$)
<code>apply_H_emb_sp(D_vec,Lmbdac,params)</code>	同一の演算子を、密行列を作らず $\hat{H}_{\text{emb}} \Phi$ として疎行列積の和で直接評価 ($B \geq 6$)
<code>prepare_sparse_params(ga,U_final,U_freq)</code>	演算子を疎のまま保持する軽量 <code>params</code> (<code>tdvp_helpers.prepare_full_params</code> の疎行列版)
<code>rhs_paper(t,y,params)</code>	ODE 右辺。 $i\partial_t \Phi = \hat{H}_{\text{emb}} \Phi$ 、 $\partial_t n(\omega) = -i\omega[RR^\dagger, n(\omega)]$ (式 (16),(17) に対応)。呼び出し元: <code>run()</code> 内の <code>solve_ivp</code> から毎ステップ (棄却ステップ含め) 自動的に呼ばれる。目的: 時間発展方程式の心臓部、 dy/dt を返すだけ
<code>save_static/load_static/get_static</code>	静的解 $(n_0, \Lambda, D, \Lambda^c, R, \text{fdagger}, \text{ffdagger})$ の <code>static_cache/</code> への保存・復元・キャッシュ判定
<code>save_run(rec,B,U_i,U_f)</code>	TD 結果 <code>rec</code> を <code>data/B{B}/</code> へ保存
<code>solve_static(B,U_i,T_gGA)</code>	<code>GA.optimize_selfc</code> を呼ぶ薄いラッパー (δU 間で使い回すため分離)。呼び出し元: <code>run(ga=None)</code> の内部、または <code>get_static</code> (キャッシュ無しの場合) から。目的: クエンチ「前」の平衡状態 ($t = 0$ の出発点) を作る
<code>run(B,U_i,U_f,...)</code>	1 本のクエンチの全工程: ゲージ回転で Λ_{eq} を構築 $\rightarrow n_0(\omega) = f(\omega R_0 R_0^\top + \Lambda_{\text{eq}}) \rightarrow \text{solve_ivp}$ (既定 DOP853, 適応刻み) $\rightarrow$ 各ステップで <code>observe()</code> が $d, E, \Delta E/E , F_2, \sqrt{D^\dagger D}$, <code>ghost split</code> , <code>eig Λ^c</code> を記録。呼び出し元: <code>run_fig2a.py</code> 等の各実行スクリプトから直接。目的: 「クエンチ 1 本を丸ごと実行して結果を返す」最上位のエントリーポイント

tdvp_helpers.py (ソルバーの部品)

関数	説明・対応式
----	--------

`setup_frequency_grid(N_freq)` Gauss-Legendre 求積点 $\times$ 半円 DOS $\rho(\omega) = \frac{2}{\pi}\sqrt{1-\omega^2}$ でバス周波数格子 (ω_f, w_f, ρ_f) を生成
`pack_state(Phi, n_omega)` ODE 状態ベクトル $y = (\text{Re}\Phi, \text{Im}\Phi, \text{Re}n(\omega), \text{Im}n(\omega))$ と $(\Phi, n(\omega))$ / `unpack_state` の相互変換
`prepare_full_params(ga_obj, nphysorb, nqspo, nfreq)` 演算子 nqz , 群速度 v_g スクから読み込み、 $M_D, M_{\Lambda^c}, \hat{H}_{\text{const}}$ 等 $\hat{H}_{\text{emb}}$ 組立に必要な全行列を事前計算して params 辞書にまとめる ($B \leq 5$ 密行列版。ここが唯一のディスク I/O)
`compute_fdaggerc_sp(Phi, params)` $\langle f^\dagger c \rangle$ (空間軌道基底、スピン対称化済み) を計算。_sp は sparse ではなく spatial の意
`compute_ffdagger_sp(Phi, params)` $\langle f^\dagger f \rangle = I - \text{hole}$ を計算
`solve_lambda_from_F2(ffd_target, R, omega)` $\int d\omega f(\omega) R R^\dagger + \Lambda = \text{ffd_target}$ を満たす Λ を least_squares で求める ($t=0$ の初期条件 Λ_{eq} 取得専用)

gga_static_solver.py (静的ソルバー、クラス GA)

メソッド	説明・対応式
<code>__init__(U, nghost, nphysorb, nfreq, ...)</code>	データを保存し edSolver (ed_solver.py) を生成、演算子 npz を準備。 $H_list = \text{cr.generate_orthonormal_basis}(nqspo)$ (実対称行列の正規直交基底) も構築
<code>dos_sc(x)</code>	半円 DOS $\rho(x) = \frac{2}{\pi}\sqrt{1-x^2}$ (ベーテ格子、半バンド幅 = 1)
<code>root_mu_Hemb(mu)</code> <code>calc_mu_Hemb()</code>	/ $\hat{H}_{\text{emb}}$ に μ を直接含め、粒子数を合わせる μ を求根 (root_mu_Hemb は残差、calc_mu_Hemb はラッパー)
<code>calc_Lmbdac(Lmbda, Delta)</code>	$\Lambda^c = -\Lambda - dF_{1-\Delta}[M]$ (実対称 Loewner、cr.dF 経由) — 式 (19) の実装
<code>solve_Hemb()</code>	ed_solver.solve_Hemb を呼び $\hat{H}_{\text{emb}}$ を対角化する薄いラッパー
<code>fix_gauge(R, Lmbda, ...)</code>	Λ^c を対角化する基底へ回転し、 D, R の符号・並び順を固定するゲージ固定 ($W = \text{permmat} \cdot \text{phasemat} \cdot \text{transmat}^T$)
<code>optimize_selfc(rinit, lambda0, ...)</code>	静的 GA 方程式の自己無撞着ループ本体。外側ループで R, Λ, μ を更新、内側で $F_1 = \langle f^\dagger c \rangle - R^T g(\Delta) = 0$, $F_2 = \langle f^\dagger f \rangle - \Delta = 0$ を cost_func 経由 least_squares で解く。呼び出し元: <code>td_gGA_solver.solve_static()</code> から (b7_ladder.py 等では直接)。目的: 与えられた U で「これ以上動かない」平衡解 (R, Λ, Φ) を見つける、静的計算の本体そのもの
<code>cost_func(xinit)</code> <code>_fix_degenerate_phi(Delta, $\hat{H}_{\text{emb}}$)</code>	optimize_selfc 内の least_squares に渡す F_1, F_2 残差ベクトル $\hat{H}_{\text{emb}}$ が縮退している場合、縮退部分空間内で $\ \langle f^\dagger f \rangle - \Delta\ $ を最小化する $ \Phi\rangle$ を選び直す (COBYLA)
<code>calc_Z(Lmbda, R)</code>	準粒子繰り込み因子 Z の一般 n_{qspo} 閉形式 ($\propto (\sum_i r_i^2 \prod_{j \neq i} \lambda_j)^2$ 型)
<code>make_Hqp(x)</code>	準粒子ハミルトニアン $\hat{H}_{\text{qp}} = \omega R R^\dagger + \Lambda$ の構築

<code>calc_Delta()</code>	$\Delta = \int \rho(\omega) f(\omega R R^\top + \Lambda) d\omega$ (<code>scipy.integrate.quad</code>) — 式 (5)
<code>calc_D()</code>	$D = [\Delta(1 - \Delta)]^{-1/2} R^\top B$ (B は運動エネルギー行列) — 式 (18)
<code>calc_Eqp()</code>	準粒子エネルギー $E_{qp} = 2 \text{Tr}[R R^\dagger B]$
<code>optimize_selfc_metallic</code> <code>/ _new / _offdiag</code> <code>/ _routeA,</code> <code>cost_func_Delta,</code> <code>calc_phase_diagram</code>	[削除済み] 2026-07-13 の整理でいずれも呼び出し元ゼロと確認し削除 (経緯は README.md 「補足: ファイル名について」 参照)

ed_solver.py (クラス edSolver、埋め込みハミルトニアンの厳密対角化)

メソッド	説明・対応式
<code>__init__</code>	軌道数 (物理・ゴースト・バス) を設定し <code>build_creation_ops</code> → <code>build_op_prods</code> で演算子一式を準備
<code>build_creation_ops()</code>	Fock 空間の全配置に対し、フェルミオン生成演算子 $f_o^\dagger$ (ビット文字列表現、Jordan-Wigner 符号付き) を疎行列で構築
<code>build_op_prods(FH_list)</code>	一体演算子積 (<code>phys-phys</code> , <code>bath-phys</code> , <code>phys-bath</code> , <code>bath-bath</code>) ・ 二体演算子積 ・ $\hat{S}^2$ を半充填ブロックのみ npz 保存 (B ごとに同名上書き)
<code>set_parameters(D,H1,Lc,U)</code>	D, H_1, Λ^c をスピン空間に複製し、二体相互作用テンソル $V_{2E}(U)$ を構築
<code>read_prod_1e(facMat,...)</code>	ディスクの一体演算子積を読み込み、係数行列 <code>facMat</code> との線形結合、または期待値 $\text{facMat}^\top(\text{op})\text{facMat}$ を計算
<code>build_Hemb(D,H1,Lc,U)</code>	$\hat{H}_{\text{emb}} = \text{一体項}(H_1, D, \Lambda^c) + \text{二体項}(U) + \text{スピンペナルティ } 10\hat{S}^2$ (一重項選択) を疎行列で構築
<code>solve_Hemb(X,U)</code>	$\hat{H}_{\text{emb}}$ を <code>primme.eigsh (which='SA')</code> で疎対角化し基底状態を取得、密度行列・局所エネルギー・二重占有を計算。呼び出し元: <code>GA.solve_Hemb()</code> 経由で <code>optimize_selfc</code> の毎イテレーションから。目的: 今の D, Λ^c での埋め込み多体基底状態 $ \Phi\rangle$ を得る (ED 部分の本体)
<code>sanity_check()</code>	半充填・一重項・基底状態であることを検証 (不一致なら例外)
<code>update_eig_vec(Phi)</code>	縮退空間内で選び直した Φ に固有ベクトルを更新し密度行列を再計算
<code>calc_density_matrix()</code>	1 粒子密度行列 $\langle c^\dagger c \rangle, \langle f^\dagger c \rangle, \langle f^\dagger f \rangle$ を計算 (<code>lspin_sym</code> ならスピン対称化)
<code>calc_double_docc(idx)</code>	二重占有数 $\langle n_\uparrow n_\downarrow \rangle$ を計算
<code>calc_Eloc()</code>	局所一体エネルギー $E_{1\text{loc}}$ ・ 二体エネルギー $E_{2\text{loc}}$ を計算

convenience_routines.py (Lanata 系の汎用行列補助関数)

関数	説明・対応式
calc_Fermi0(x,T)	Fermi 関数 ($T = 0$ では階段関数 $\theta(-x)$)
funcMat(H,function,T)	エルミート行列 H の汎用行列関数 $f(H) = Uf(\text{diag})U^\dagger$ (eigh ベース)
generate_orthonormal_basis(N)	$N \times N$ 実対称行列の Frobenius 内積に関する正規直交基底 (GA.__init__ の H_list 構築に使用)
Hermitian_list(N)	$N \times N$ エルミート行列の正準基底 (対角 + 実対称非対角 + 虚反対称非対角)
realHcombination(x,H_list)	実係数ベクトル x と基底 H_list の間の相互変換: $H = \sum_n x_n [H_list]_n \leftrightarrow x_n = \text{Tr}[H_list_n H]$
inverse_realHcombination	
dF(A,H,function,d_function)	Downer の定理による行列微分 $Df(A)(H) = f^{[1]}(A) \circ H$ (GA.calc_Lmbdac が使用)
calc_C(H,T)	密度行列 $C = f_{\text{Fermi}}(H)$ (funcMat 経由、多用される中核関数)
denRm1(x) / denR(x) / ddenRm1(x)	$[x(1-x)]^{\pm 1/2}$ とその微分 $\frac{d}{dx}[x(1-x)]^{1/2} = \frac{0.5-x}{[x(1-x)]^{1/2}}$
duplicate_in_spin_space(A)	軌道空間の行列 A をスピン空間へ複製: $\tilde{A}_{2i,2j} = \tilde{A}_{2i+1,2j+1} = A_{ij}$
spin_symmetrize(A)	上の逆操作、スピニアップ・ダウンを平均: $A_{ij}^{\text{sym}} = \frac{1}{2}(A_{2i,2j} + A_{2i+1,2j+1})$
complexHcombination,	[未使用] 呼び出し元なし (複素版基底変換・行列ブロック分割・数値
inverse_complexHcombination,	微分の検証用ヘルパー、Lanata 系の遺産)
get_blocks, dF_numerical	
calc_offdiag_dd	[未使用・要修正候補] 未定義変数 (D, fdaggerc, Lmbdac, self, optimize) を参照しており、呼び出すと即エラーになる。どこからも呼ばれていない

lattice.py

[未使用] 一般的な tight-binding 格子クラス (2D 正方・三角・グラフェン・3D 立方対応、Fourier 変換・DOS 計算メソッド付き)。gga_static_solver.py が `from lattice import *` しているが、実際に呼ばれるメソッドは一切ない。本パッケージが使う半円形 DOS は `GA.dos_sc` に別途ハードコードされており、このファイルの機能は使われていない。